Unidad 7 — NoSQL y nuevas formas de almacenamiento (Food Store)

Origen: PostgreSQL 18.6 `foodstore_tp` (solo lectura). **Destino:** MongoDB 4.4.32 `foodstore_u7.pedidos` (shell `mongo.exe`). **Subconjunto:** 5 pedidos reales con líneas completas: IDs **1,4,5,6,18** (15 líneas). Sin PII ni credenciales en entrega; scripts separados: `migracion_u7.js`, `validacion_u7.js`, `consultas_mongodb.js`, `consultas_postgresql.sql`.

1. Modelo documental y migración

Tablas/claves PostgreSQL (verificadas): `pedidos` PK `id_pedido` (FK `id_cliente -> clientes`, `id_sucursal -> sucursales`, `id_empleado_repartidor -> empleados`, campos fecha/estado/tipo_entrega/total/datos_envio/activo); `detalle_pedido` PK `id_detalle` (FK `id_pedido -> pedidos` ON DELETE CASCADE, `id_producto -> productos`, UNIQUE (id_pedido,id_producto), cantidad/precio_unitario/subtotal>0); `clientes` PK `id_cliente`; `productos` PK `id_producto`. Relaciones 1-N: clientes -> pedidos, pedidos -> detalle_pedido, productos -> detalle_pedido.

Esquema colección `foodstore_u7.pedidos` (1 doc = 1 pedido): `_id=id_pedido` (conservado), `id_pedido`, `fecha_pedido`(ISODate), `estado`, `tipo_entrega`, `id_sucursal`, `id_empleado_repartidor`, `total`(NumberDecimal), `activo`, `datos_envio` (objeto embebido), `cliente` (objeto embebido: `id_cliente, cliente_ref, fecha_registro, activo` sin nombre/apellido/email/teléfono/dirección), `detalle[]` (arreglo embebido: `id_detalle,id_producto,producto_nombre,cantidad,precio_unitario,subtotal` NumberDecimal), `origen`.

Migración: precheck `count=0`, sin datos previos; `insertMany` 5 docs desde `foodstore_tp` (solo lectura). Resultado: `INSERT_OK insertedCount=5`, IDs [1,4,5,6,18]. Importes históricos como `NumberDecimal` (sin redondeo).

Integridad conservada (2 reglas): (1) Monetaria histórica: por línea `subtotal=cantidad*precio_unitario` y `total=Σ subtotales` (valores copiados, sin recalcular con precios actuales). (2) Referencial/completitud: IDs originales conservados, líneas reales completas, estado/tipo válidos, cantidades e importes > 0, correspondencia con origen.

Justificación embebido: `detalle` embebido para leer pedido completo en una sola operación (acceso frecuente, cardinalidad acotada, sin `$lookup`); `cliente` como snapshot esencial sin PII; precios históricos en línea para trazabilidad.

2. Validación de migración (resultados reales)

Verificación	Resultado
Conteo documentos	5 (OK)
IDs	[1,4,5,6,18] (OK)
Líneas por pedido	3,3,3,3,3 (OK, 15 líneas)
subtotal = cantidad × precio_unitario (todas)	true (OK)
total = Σ subtotales	true (1:39676.55, 4:10547.69, 5:24590.33, 6:21834.90, 18:23949.34)
Duplicados	0 (OK)

Comparación directa 15 líneas (PG vs Mongo): campos `id_detalle,id_producto,cantidad,precio_unitario,subtotal` por pedido -> **15/15 coinciden exactamente** (`diff_count=0`). Evidencia: `evidencias_t08b_lineas.txt`.

Privacidad (verificada, no asumida): consulta real comparó `datos_envio` en PG y Mongo (IDs 1,4,5,6,18): valores idénticos (`{"ciudad":"Mendoza","direccion":"Calle 1/4/5/6/18"}`), provienen de `foodstore_tp` con formato genérico de prueba/sintético, sin datos personales reales identificables en los campos verificados. En Mongo, `cliente` contiene solo `[activo,cliente_ref,fecha_registro,id_cliente]` (5/5, sin nombre/apellido/email/teléfono/dirección; `sin_pii=true`). No se modificó `foodstore_tp`. Evidencias: `evidencias_t08c_privacidad.txt`, `evidencias_privacidad_datos_envio.txt`.

3. Consultas y comparación PostgreSQL / MongoDB

MongoDB (a–d): a) Todos (id,estado): (1,cancelado),(4,en_camino),(5,en_preparacion),(6,entregado),(18,pendiente). b) estado="entregado": [6]. c) producto `id_producto=8` en detalle: [4,5,6]. d) `total > 20000` (umbral **20000**): [1,5,6,18]. *Justificación umbral:* separa ticket alto del subconjunto (4/5 lo superan, pedido 4=10547.69 queda fuera), verificando filtro numérico sobre `NumberDecimal` de forma no trivial.

Consulta	Cant. PostgreSQL	IDs PostgreSQL	Cant. MongoDB	IDs MongoDB	Coinciden
b) estado = entregado	1	{6}	1	[6]	**Sí**
c) contiene id_producto=8	3	{4,5,6}	3	[4,5,6]	**Sí**

Diferencias: ninguna en cantidad ni IDs; no se requirió corrección de modelo/consultas. Consultas ejecutadas con `consultas_mongodb.js` y `consultas_postgresql.sql` (subconjunto, solo lectura).

4. Verificación de afirmaciones de IA y conclusión

Consulta IA (Kilo Code): comparación PostgreSQL vs MongoDB para pedidos de Food Store considerando modelo relacional (claves/constraints), acceso frecuente a pedidos completos y coherencia de cantidades/precios históricos.

Afirmación 1: “Embeber detalle favorece leer el pedido completo en una sola consulta”. **Contraste:** Doc. oficial MongoDB

(Embedded Data Models: relaciones “contains”/acceso conjunto, una lectura, atomicidad a nivel documento, https://www.mongodb.com/docs/manual/data-modeling/embedding/). **Laboratorio:** en `foodstore_u7.pedidos` las consultas a–d/lectura completa se resuelven con `find()` sin `$lookup`; en PostgreSQL requiere JOINs. **Confirmada.**

Afirmación 2: “PostgreSQL garantiza mejor integridad referencial y de importes vía constraints/transacciones; en MongoDB la coherencia debe asegurarse en modelo/migración”. **Contraste:** Doc. oficial MongoDB (modelado embebido/relaciones: consistencia de referencias y campos calculados recae en diseño/aplicación; https://www.mongodb.com/docs/manual/data-modeling/embedding/). **Laboratorio:** PostgreSQL define PK/FK, UNIQUE y CHECKs (cantidad/precio/subtotal>0, total>=0); en MongoDB 4.4 se preservó con `NumberDecimal` y se validó `subtotal=cantidad*precio` y `total=Σ subtotales` (5/5 true, sin duplicados). **Confirmada.**

Conclusión (≤`200 palabras`)

Para el subconjunto “pedidos y detalle” de Food Store conviene una **combinación**: mantener **PostgreSQL** como fuente transaccional autoritativa y usar **MongoDB** como vista documental para lectura de pedidos completos. (1) **Acceso:** el pedido se consulta como unidad con sus líneas; el modelo embebido permite recuperarlo en una sola operación `find()` sin joins, coherente con la guía de embedding de MongoDB y verificado en las consultas a–d. (2) **Integridad y coherencia histórica:** PostgreSQL impone FKs, CHECKs y unicidad que protegen cantidades, precios y totales en origen; en MongoDB 4.4 la consistencia (`NumberDecimal`, `subtotal=cantidad*precio_unitario`, `total=Σ subtotales`) debe garantizarse en migración/validación, comprobado en los 5 documentos (`total_match_all` y `lineas_calc_all` verdaderos, 15/15 líneas exactas, sin duplicados). (3) **Trazabilidad:** conservar `_id=id_pedido` e IDs originales permitió verificar equivalencia exacta entre motores (b: {6}; c: {4,5,6}). Así, MongoDB optimiza lecturas documentales sin reemplazar las garantías relacionales del sistema principal.

Archivos en `food_store_unidad7b2/`: `informe_u7.pdf/.md`, `README.md`, `migracion_u7.js`, `validacion_u7.js`, `consultas_mongodb.js`, `consultas_postgresql.sql` y evidencias (.txt). Sin modificar `foodstore_tp`, otras bases MongoDB ni unidades anteriores. Sin commit/push.